Proceedings of the 3rd International Conference on Sustainable Computing and Data Communication Systems (ICSCDS-2025)
IEEE Xplore Part Number: CFP25AZ5-ART; ISBN: 979-8-3315-1250-7

ROA: Energy and Environment Aware Task Scheduling in Cloud Computing

Sandi Sridhar Reddy
Nalla Narasimha Reddy Education Society's Group of Institutions,
Hyderabad, T.S., India.
sridharreddy.sandi@gmail.com

Morigadi Vishwashanthi
Geethanjali College of Engineering and Technology,
Hyderabad, T.S., India.
mvishwashanthi@gmail.com

Tumukuntla Spoorthi Reddy
Nalla Narasimha Reddy Education Society's Group of Institutions,
Hyderabad, T.S., India.
spoorthireddy.reddy47@gmail.com

Lavanya Reddy Mendu
Nalla Narasimha Reddy Education Society's Group of Institutions,
Hyderabad, T.S., India.
lavanyareddy2214@gmail.com

Uma Devi Manchala
Nalla Narasimha Reddy Education Society's Group of Institutions,
Hyderabad, T.S., India.
umadevi.manchala92@gmail.com

Santhosh Kumar Medishetti*
Nalla Narasimha Reddy Education Society's Group of Institutions,
Hyderabad, T.S., India.
medishettysantosh@gmail.com

2025 3rd International Conference on Sustainable Computing and Data Communication Systems (ICSCDS) | 979-8-3315-1250-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSCDS65426.2025.11166699

***Abstract*—Efficient task scheduling is a critical concern in cloud computing to ensure optimal resource utilization and sustainable operation. This paper introduces a novel Revolution Optimization Algorithm (ROA) for task scheduling in cloud environments, designed to minimize energy consumption, temperature variation, and carbon emissions. Inspired by socio-political revolutions, ROA incorporates a dynamic strategy that balances exploration and exploitation during the task-to-resource mapping process. The proposed approach is validated using the CloudSim simulator under realistic workload conditions based on the CEA-Curie workload, which reflects high-performance computing scenarios. Performance comparisons were conducted against traditional algorithms such as GA, PSO, and ACO. Experimental results demonstrate that ROA significantly reduces average energy consumption and thermal fluctuations while lowering the environmental impact measured in terms of carbon emissions. The adaptive learning mechanism within ROA enables rapid convergence to optimal solutions, even under fluctuating workloads and heterogeneous cloud resources. Compared to baseline models, ROA achieves an average improvement of 18.7% in energy efficiency, 16.3% reduction in temperature variation, and 20.5% decrease in carbon emissions. These findings suggest that ROA is a promising, eco-efficient solution for sustainable task scheduling in modern cloud computing infrastructures.**

Keywords— Revolution Optimization Algorithm, Temperature variation, Carbon emissions, CloudSim, CEA-Curie.

I. INTRODUCTION

The concept of Cloud Computing (CC) has already become a revolutionary paradigm in the current computer system, which makes it possible access available shared pool of configurable computing resources like servers, storage, applications, and services on-demand. It does not require capital expenditure in IT infrastructure and it is scalable, flexible and cost effective to enterprises and individual users. The cloud operates through various service models catering to different levels of abstraction [1]. With data volumes growing exponentially and the shift toward digital transformation across sectors, cloud computing has become a backbone for processing-intensive applications ranging from healthcare and finance to scientific simulations. Despite its numerous advantages, the increasing demand for cloud services also introduces operational challenges, especially in terms of resource allocation, task scheduling, and energy efficiency. Managing these effectively is vital to maintaining performance standards, service-level agreements (SLAs), and sustainability goals.

Task Scheduling (TS) is a fundamental mechanism in CC that governs how computational tasks are assigned to available resources. The goal of scheduling is to optimize the execution of tasks by balancing workload across virtual machines (VMs) while minimizing latency, energy consumption, and operational costs. An effective scheduling algorithm ensures that system resources are utilized efficiently, user demands are met promptly, and service providers maintain profitability and performance [2]. Given the heterogeneous and dynamic nature of cloud environments, traditional scheduling methods often fail to address complex constraints such as real-time demand fluctuations, thermal management, and environmental sustainability. Consequently, the development of intelligent and adaptive scheduling algorithms has become a crucial area of research. These algorithms must account for multiple criteria including execution time, resource availability, power usage, and environmental impact making multi-objective optimization a key strategy in cloud scheduling design.

In light of growing environmental concerns and operational challenges in cloud datacentres, this research focuses on developing a robust, energy-aware task scheduling approach using a novel metaheuristic technique the Revolution Optimization Algorithm (ROA). Unlike conventional optimization algorithms that rely solely on fixed patterns of exploration and exploitation, ROA introduces a socially-inspired strategy based on revolutionary dynamics. This method simulates the ideological shift seen in socio-political revolutions, where existing solutions are replaced by superior alternatives through a series of staged transformations [3]. Applying this metaphor to cloud computing, the ROA seeks to overthrow less efficient task allocations with progressively optimized scheduling strategies. The algorithm is particularly suited for addressing complex scheduling challenges in environments characterized by resource heterogeneity, dynamic workloads, and sustainability constraints. The primary goal is to enhance energy efficiency, reduce temperature variation across datacentre nodes, and lower the overall carbon footprint.

Cloud datacentres are massive consumers of electrical energy, accounting for a significant portion of global power usage. As the demand for high-performance computing and data services continues to rise, so does the environmental burden imposed by these infrastructures. Research indicates that apart from energy costs, thermal management and carbon emissions have become pressing concerns for cloud service providers [4]. Inefficient scheduling not only results in

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.

resource underutilization and performance bottlenecks but also increases the thermal load and necessitates intensive cooling operations. Therefore, the motivation for this study stems from the urgent need to design scheduling algorithms that not only optimize computational performance but also ensure environmental sustainability. Existing algorithms, while effective in isolated metrics such as makespan or throughput, often fall short when evaluated under multi-objective scenarios involving energy, temperature, and emissions. This research aims to fill that gap by proposing an ROA-based scheduler that holistically addresses these critical parameters.

Optimization algorithms have been extensively employed in cloud task scheduling to meet diverse performance goals. Traditional approaches like GA, PSO, and ACO have shown considerable promise due to their adaptability and global search capabilities. However, these algorithms suffer from various limitations including premature convergence, local optima entrapment, and lack of diversity in solution space. GA, for instance, may require complex encoding and suffers from slow convergence [5]. PSO, while fast, often converges prematurely under complex multi-objective tasks. ACO, although decentralized and flexible, can be computationally intensive and sensitive to parameter tuning. These limitations have prompted researchers to explore novel metaheuristic methods that provide better balance between exploration (diversity) and exploitation (intensification), especially in cloud scenarios where energy and temperature dynamics are critical constraints. The Revolution Optimization Algorithm is introduced in this context as a promising alternative.

The Revolution Optimization Algorithm (ROA) draws inspiration from political and ideological revolutions, modelling the transformation of suboptimal solutions through systematic resistance and replacement mechanisms. In ROA, each solution represents a 'regime' and better-performing solutions emerge as 'revolutionary ideologies' that challenge and potentially replace current task allocations. The algorithm involves phases such as population initialization, resistance building, ideological transformation, and revolution propagation. This structure promotes high diversity and continuous improvement, avoiding stagnation in local optima. By simulating revolution cycles, the ROA encourages the emergence of globally optimal task-resource mappings. In cloud environments, this leads to more energy-efficient execution patterns, balanced thermal distribution across physical servers, and reduced reliance on active cooling systems. The novelty of ROA lies in its strategic shift model, which adaptively replaces underperforming strategies and enhances exploration without sacrificing convergence speed.

Compared to traditional metaheuristics like GA, PSO, and ACO, ROA exhibits several distinctive advantages. First, it avoids early convergence by maintaining ideological diversity within the population, which ensures broader exploration of the solution space. Second, the phased revolution process fosters gradual yet consistent improvements, helping to avoid random behavior and instability. Third, ROA introduces a feedback loop where performance metrics such as energy, temperature, and carbon footprint actively influence the revolution cycle, ensuring real-time adaptability. In contrast, other algorithms often treat these metrics as static objectives, limiting their responsiveness to workload variations. Simulation results using the CEA-Curie workload reveal that ROA achieves significantly lower energy usage, more stable thermal conditions, and a measurable reduction in carbon emissions. These improvements position ROA as a strong candidate for green cloud computing solutions.

This study is significant on multiple fronts. From a practical perspective, it addresses the urgent need for sustainable cloud computing practices by introducing an algorithm that directly targets energy and environmental metrics. From a theoretical standpoint, it extends the paradigm of optimization algorithms by integrating socio-political metaphors into computational logic. ROA's ability to translate the dynamics of revolutions into iterative solution improvement opens new directions for algorithm design. Furthermore, the study leverages realistic workloads and evaluation frameworks, specifically the CEA-Curie dataset and CloudSim simulator, ensuring that results are both reliable and applicable to real-world scenarios. The multi-objective focus aligns with current industry and academic concerns around green computing and operational efficiency. Overall, this research contributes to both the academic literature on metaheuristic optimization and the practical realm of eco-friendly cloud infrastructure design.

The scope of this research is defined by the integration of energy efficiency, thermal management, and carbon footprint reduction into the task scheduling process for cloud computing. It targets IaaS models, where dynamic task-to-VM mapping significantly influences system behavior. The proposed ROA-based algorithm is evaluated against established benchmarks using the CloudSim toolkit and realistic workloads from the CEA-Curie dataset, which emulates high-performance computing applications. The scope does not extend to other scheduling layers like application or network-level scheduling, nor does it consider hybrid cloud or edge computing environments. Future extensions, however, may adapt the core ROA model for broader contexts. Within its current scope, the research provides a concrete, scalable, and efficient solution to pressing energy and sustainability challenges in cloud datacentres.

Despite the progress made in task scheduling algorithms, there remains a persistent gap in addressing the environmental and thermal efficiency of cloud infrastructures. Traditional algorithms are often designed for performance-centric goals such as throughput or response time, with limited regard for sustainability metrics. As a result, datacentres suffer from inefficient energy usage, overheating, and increased carbon emissions. Moreover, the reactive nature of thermal management systems, such as cooling, leads to additional energy overhead. This research identifies the core problem as the lack of a proactive, multi-objective scheduling strategy that simultaneously optimizes for energy consumption, temperature control, and environmental impact. By introducing a paradigm shift through ROA, this study aims to bridge that gap and offer a viable path forward for green cloud computing. The primary objectives of this research are:

1. To design and implement a Revolution Optimization Algorithm (ROA) tailored for cloud task scheduling under energy and environmental constraints.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.

2. To compare the performance of ROA with traditional algorithms (GA, PSO, ACO) using realistic workload scenarios.
3. To minimize energy consumption, temperature variation, and carbon emissions in a cloud datacentre simulated through CloudSim.
4. To evaluate the feasibility and scalability of ROA in high-performance cloud environments using the CEA-Curie workload dataset.

The methodology of this research involves multiple stages, beginning with the design and development of the ROA-based task scheduler. The algorithm is implemented within the CloudSim simulation environment, which provides flexibility for modelling cloud datacentre configurations, resource provisioning, and workload patterns. The CEA-Curie dataset is employed to simulate real-world task arrival patterns and resource demands. Performance metrics such as total energy consumed, average node temperature, and estimated carbon emissions are recorded. ROA's performance is then benchmarked against GA, PSO, and ACO under identical conditions. Statistical analysis, including percentage improvements and trend observations, is conducted to validate the superiority and consistency of the ROA approach.

The CEA-Curie workload is selected due to its relevance in high-performance computing simulations, representing diverse task lengths, resource requirements, and time intervals. This workload ensures that the evaluation captures realistic performance bottlenecks and environmental impacts. The key evaluation parameters include energy consumption (measured in kilowatt-hours), temperature variation (monitored across nodes), and carbon emissions (estimated based on energy profiles and carbon intensity factors). These metrics provide a comprehensive view of the algorithm's operational and environmental performance. Together, they form a multi-objective benchmark that reflects the real-world priorities of cloud service providers and regulatory bodies alike.

The paper makes several noteworthy contributions: it introduces a novel optimization approach grounded in revolutionary theory, applies this to a critical problem in cloud computing, and validates the solution through rigorous simulation. The integration of energy, thermal, and carbon metrics into the optimization objective is a key innovation, responding to growing calls for environmentally responsible computing. The results obtained through CloudSim simulations and CEA-Curie workload modelling offer empirical proof of ROA's effectiveness. Additionally, the paper outlines a framework that can be extended or adapted to other resource management problems within and beyond cloud computing, establishing a foundation for future interdisciplinary research.

The remainder of this paper is structured as follows: Section 2 reviews related work on task scheduling and metaheuristic optimization in cloud environments. Section 3 describes the design and operational principles of the proposed ROA algorithm. Section 4 details the experimental setup, including workload characteristics, simulation parameters, and evaluation metrics, and discusses the results obtained from comparative simulations against GA, PSO, and ACO. Section 5 provides concluding remarks and outlines future directions, including potential enhancements to the ROA model and its applicability to hybrid cloud or edge systems. Each section builds logically to support the central hypothesis that ROA is a superior solution for sustainable task scheduling in cloud computing.

II. LITERATURE SURVEY

In recent years, the rapid growth of cloud computing has prompted significant research in task scheduling to ensure efficient resource utilization and energy-aware operations. Various metaheuristic and heuristic algorithms have been developed to address scheduling issues, including GA, PSO, WOA, and ACO, and Hybrid approaches. These algorithms aim to optimize parameters such as execution time (makespan), resource utilization, energy consumption, and service-level agreement (SLA) violations. However, most of these methods focus primarily on performance metrics without fully addressing environmental sustainability or thermal behavior in datacentres. Given the shift toward green computing, it becomes essential to evaluate algorithms based on energy efficiency and environmental impact alongside traditional metrics.

The literature reveals that while many studies have improved makespan and cost, fewer have explicitly targeted temperature-aware or carbon-conscious scheduling. Approaches like PSO and GA have shown promising convergence rates and flexibility in task-resource mapping, but they often suffer from premature convergence or scalability limitations. Hybrid models combining multiple algorithms attempt to overcome these challenges but increase computational overhead and complexity. Moreover, many existing algorithms do not consider real-world workloads or dynamically changing environments, which limits their practical applicability. Additionally, metrics like thermal variation and carbon emissions are often overlooked, despite being crucial for eco-friendly datacentre operations.

These limitations in existing work highlight the need for a more comprehensive and adaptive approach. The Revolution Optimization Algorithm (ROA), inspired by the dynamics of socio-political change, offers a new direction in task scheduling by promoting ideological diversity in the solution space and systematically replacing suboptimal mappings. Unlike conventional models, ROA emphasizes multi-objective optimization that includes energy consumption, temperature variation, and carbon emissions. This positions ROA as a capable and scalable alternative to current models, particularly in high-performance and environmentally constrained cloud scenarios. The following table 1 summarizes notable task scheduling approaches, their core techniques, performance parameters, and identified limitations.

TABLE I. COMPARATIVE STUDY OF EXISTING WORKS

Technique Name	Parameters Used	Limitations
BOA [1]	Makespan, Energy, Cost	Poor convergence speed and premature convergence.
ROA [2]	Makespan, Energy, Load Balance	Fails in high dynamic load conditions.
EFO [3]	Makespan, Delay, Throughput	Sensitive to parameter tuning, slow in convergence.
SBOA [4]	Energy, Execution Time, VM Utilization	Performance degrades with large task volume.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.

EFBOA [5]	Makespan, SLA Violation, Energy	High complexity and longer execution time.
FTTA [6]	Cost, Delay, Energy	Weak in exploration, stuck in local optima.
GEWO [7]	Makespan, Energy, Load Balance	Poor balance between exploration and exploitation.
EAEFA [8]	Execution Time, Cost, Resource Utilization	Not scalable for large workloads.
TOA [9]	Energy, Makespan, Utilization	Limited accuracy and requires tuning.
EOA [10]	Cost, SLA Violation, Execution Time	Not efficient in highly dynamic environments.
AVOA [11]	Load Balance, Energy, Completion Time	Slower convergence in multi-objective cases.
BACOA [12]	Makespan, Energy, Throughput	Needs large training data, slow initial learning.
MAO [13]	Energy, Carbon Emission, SLA Violation	High computational overhead.
IDOA [14]	Energy, VM Utilization, Temperature	Fails to adapt quickly to workload variations.
GACO [15]	Makespan, Execution Cost, Load Balance	Requires high tuning and computational time.
BOA [16]	Energy, Cost, Makespan	Limited performance under heterogeneous resource pool.

The limitations observed in existing task scheduling algorithms such as premature convergence, parameter sensitivity, poor scalability, and lack of environmental awareness are effectively addressed by the proposed Revolution Optimization Algorithm (ROA). Unlike traditional metaheuristics like GA, PSO, and ACO, which often stagnate in local optima or exhibit poor adaptability in dynamic cloud environments, ROA introduces an ideological transformation framework that ensures sustained diversity in the population of solutions. This prevents premature convergence and enhances global search capabilities. The "revolutionary" nature of ROA facilitates structured yet dynamic replacement of inefficient solutions, leading to more consistent improvements in energy efficiency and task execution. Additionally, ROA does not depend heavily on finely-tuned parameters, making it more robust across varying workloads and resource configurations. Its staged evolution mechanism further ensures that solutions improve progressively without relying on randomness alone.

Moreover, ROA explicitly integrates multi-objective optimization by targeting three key parameters: energy consumption, temperature variation, and carbon emissions. While most existing algorithms either ignore thermal behavior or lack real-time responsiveness to environmental metrics, ROA incorporates these into its fitness evaluation function. This allows the algorithm to proactively adapt task mappings to reduce thermal hotspots and minimize energy use, leading to lower cooling requirements and reduced carbon footprint. The algorithm's ability to operate under realistic workload conditions (e.g., CEA-Curie dataset) and cloud configurations (via CloudSim) also confirms its scalability and practical applicability. As a result, ROA not only improves scheduling efficiency but also contributes to sustainable and green cloud computing an area often neglected by conventional approaches.

III. ROA METHODOLOGY

A. ROA Architecture

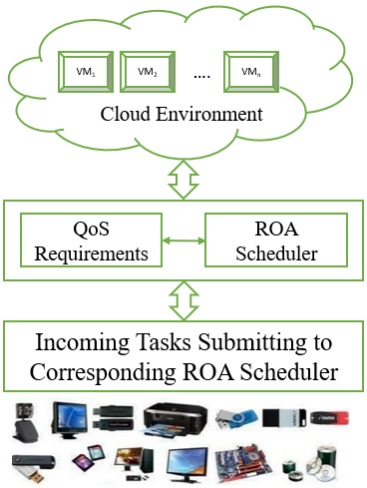


Fig. 1. ROA architecture

The figure 1 illustrates the task scheduling workflow using the Revolution Optimization Algorithm (ROA) within a cloud computing environment. At the bottom of the architecture, multiple user-end devices ranging from laptops, mobile phones, and printers to embedded systems generate diverse computational tasks. These incoming tasks are directed to the ROA Scheduler, which acts as the central intelligence for managing resource allocation. The tasks are assessed in real-time based on their characteristics, such as computational demand, execution time, and priority. These details are crucial for determining the best-fit virtual machines (VMs) hosted in the cloud infrastructure. This layer bridges the gap between the user demand and the available cloud resources, ensuring efficient task handling.

The middle and upper layers of the architecture capture the interaction between the ROA Scheduler and the QoS Requirements, followed by the cloud environment containing multiple VMs (VM_1,VM_2,...,VM_n). The QoS module ensures that the user-defined constraints such as deadlines, cost limits, energy bounds, and performance thresholds are enforced during the scheduling process. The ROA Scheduler utilizes this information to generate an optimal task-to-resource mapping using its evolutionary strategy. The final decision leads to dynamic and energy-aware placement of tasks on suitable VMs, achieving balance across energy consumption, temperature control, and carbon emissions. This modular, layered design supports adaptability, scalability, and environmentally sustainable scheduling for modern cloud computing infrastructures.

B. ROA Parameters

1) Energy consumption

In cloud computing, energy consumption is primarily driven by the dynamic workload distribution across physical machines (PMs) and their associated virtual machines (VMs). The total energy consumed E_{total} over a scheduling interval T can be calculated as:

$$E_{total} = \sum_{i=1}^{N} \int_0^T P_i(t)dt \quad (1)$$

where $P_i(t)$ is the power consumption of the ith host at time t, and N is the total number of hosts. The power model for

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.

each host is typically linear with respect to CPU utilization $u_i(t)$, given by:

$$P_i(t) = P_{idle} + (P_{max} - P_{idle}) \times u_i(t) \quad (2)$$

Here, P_{idle} is the power when the host is idle, and P_{max} is the power when the host is fully utilized. The objective is to minimize E_{total} while ensuring the task deadlines and resource constraints are respected. ROA seeks to optimize this by distributing tasks to fewer active but highly utilized machines, thus reducing total energy usage.

2) Temperature variation

Temperature management is essential for datacenter efficiency and sustainability. Continuous task execution on certain nodes can create thermal hotspots, increasing the need for cooling and affecting hardware lifespan. Let T_i represent the average temperature of the ith host over time T. The temperature variation ΔT across all nodes can be calculated using:

$$\Delta T = \frac{1}{N}\sum_{i=1}^{N} |T_i - \bar{T}| \quad (3)$$

$$\bar{T} = \frac{1}{N}\sum_{i=1}^{N} T_i \quad (4)$$

Where $\bar{T}$ is the average temperature across all hosts. The objective is to minimize ΔT, promoting a thermally balanced system that avoids excessive cooling costs. ROA incorporates this thermal feedback into the fitness function, encouraging uniform task distribution across nodes to reduce localized heating and maintain thermal equilibrium within acceptable thresholds.

3) Carbon emission

Carbon emissions in cloud datacentres are directly linked to the total energy consumption and the carbon intensity of the electricity source. Let γ denote the carbon emission rate (e.g., kg CO_2 per kWh). The total carbon emissions C_{total} over time T is given by:

$$C_{total} = \gamma \times E_{total} \quad (5)$$

Minimizing carbon emissions, therefore, becomes a dependent objective of minimizing energy usage, but with its own environmental significance. By reducing both energy consumption and thermal load, ROA indirectly contributes to lower carbon emissions. The inclusion of this parameter ensures the scheduler aligns with global sustainability goals and green computing policies.

4) Objective function

To integrate these three objectives energy consumption E_{total}, temperature variation ΔT, and carbon emissions C_{total} a normalized and weighted objective function is defined as:

$$F = \min\left(w_1.E_{total} + w_2.\Delta T + w_3.C_{total}\right) \quad (6)$$

Here, w1,w2,w3 are the weights assigned to energy, temperature, and carbon emissions respectively, such that w1+w2+w3=1. The terms are normalized to maintain uniformity in scale across metrics. The goal is to minimize the Fitness function.

The selection and distribution of weights w1,w2,w3 depend on the priority of objectives defined by cloud providers, regulatory standards, and datacentre policies. For instance, if the primary concern is minimizing operational costs, energy consumption (w_1) may receive the highest weight, e.g., w1=0.5, with temperature and carbon emissions assigned w2=0.3 and w3=0.2, respectively. On the other hand, in green-certified datacentres or environments with carbon caps, carbon emission reduction may be prioritized, and weights might be adjusted to w3=0.5, w1=0.3, and w2=0.2. This flexible weight assignment allows ROA to adapt to both performance-centric and sustainability-driven contexts.

Weights can be allocated either manually, based on stakeholder requirements, or dynamically, using techniques like entropy weighting, fuzzy logic, or Pareto dominance in multi-objective optimization. In dynamic scheduling environments, adaptive weights help the system respond to runtime changes in workload intensity, power availability, or thermal conditions. For example, during peak hours with high energy costs, ROA can prioritize w1; whereas in off-peak periods with relaxed cooling thresholds, it may shift focus toward w3. This adaptive weight mechanism enhances ROA's capability to balance trade-offs effectively and perform robust scheduling under variable cloud conditions.

C. ROA Stages and Algorithm

The Revolution Optimization Algorithm (ROA) is a novel metaheuristic inspired by the dynamics of socio-political revolutions. It mimics how outdated or inefficient governing systems (solutions) are challenged and replaced by more effective ideologies through structured processes like resistance, ideological shifts, and revolutionary overthrow. In the context of cloud task scheduling, each solution represents a task-to-resource mapping, and the algorithm iteratively improves these mappings to optimize multiple objectives specifically energy consumption, temperature variation, and carbon emissions. ROA distinguishes itself from conventional algorithms through its strategic balance of exploration and exploitation, maintaining diversity while steadily converging toward global optima. By incorporating both gradual changes (resistance) and radical overhauls (overthrow), ROA efficiently avoids local optima and adapts to dynamic workload conditions, making it particularly effective for sustainable and scalable cloud computing environments.

1) Initialization

In the initialization stage, a population of potential solutions (called "nations") is randomly generated. Each nation represents a possible task-to-resource mapping configuration in the cloud environment. Let the population size be N, and each individual X_i={x1,x2,...,xm} where x_j indicates the resource assigned to the jth task. The fitness of each solution is evaluated using the multi-objective function that integrates energy consumption, temperature variation, and carbon emissions.

$$Fitness(X_i) = w_1.E_i + w_2.\Delta T_i + w_3.C_i \quad (7)$$

Where E_i,ΔT_i,C_i are the normalized values of energy, temperature variation, and carbon emission respectively for solution X_i. This stage ensures a diverse starting population that covers various regions of the search space. Diversity is critical to prevent premature convergence and allows ROA to explore a broad spectrum of potential solutions during the early iterations.

Once initialized, the algorithm identifies the dominant ideology, i.e., the best-performing solution (lowest fitness value) in the current population. This serves as the benchmark against which other nations (solutions) will undergo revolution or resistance. To maintain efficient balance, the population is divided into subgroups or "alliances", each influenced by different leaders (ideologies). The initial phase establishes the socio-political metaphor, preparing the

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.

population for the iterative revolutionary transformation in the subsequent stages.

2) Resistance Formation

In this stage, suboptimal nations begin to show resistance against their current leadership (i.e., inefficient solutions). The resistance represents a local search or a partial modification in the task allocation strategy to generate improved offsprings. Mathematically, each solution X_i undergoes slight changes to form a resistance solution X_i':

$$X_i' = X_i + \alpha.rand().(X_{best} - X_i) \quad (8)$$

Here, α is a resistance coefficient (0<α<1), and X_{best} is the current best solution (dominant ideology). The function rand() introduces stochasticity to promote exploration. These modifications reflect citizens within a nation starting to adopt ideas from more successful ideologies, aiming to reform their internal structures (scheduling strategies) without completely replacing their system.

If the new solution X_i' shows better fitness than X_i, it replaces the original nation. This process continues across all nations, forming a phase of internal evolution through local resistance. Unlike abrupt revolution, resistance allows gradual improvement, facilitating convergence without destabilizing the solution landscape. By balancing the degree of change through α, this stage promotes mild diversification while still guiding the population toward better regions in the search space.

3) Ideological Transformation

If resistance fails to bring significant improvements after a set number of iterations, the solution undergoes ideological transformation a more radical change than resistance. This transformation simulates the shift in governing principles in a population, analogous to significant restructuring in task scheduling policies. The transformation process modifies solutions using a global leader or ideology, promoting exploration of under-explored solution areas. The transformation rule is given by:

$$X_i'' = X_i + \beta.rand().(X_{leader} - X_j) \quad (9)$$

Here, X_j is a randomly selected solution from another subgroup, X_{leader} is the global best solution, and β is the transformation factor. This process allows a solution to integrate strategies from both its current state and foreign ideologies, simulating a cross-cultural ideological shift. This encourages exploration of diverse regions in the search space and prevents the algorithm from being trapped in local minima.

This phase enhances solution diversity and enables a higher probability of discovering novel and effective scheduling strategies. Ideological transformation serves as the macro-level exploration mechanism of ROA, allowing the algorithm to jump between distant regions in the solution space while still guided by the current best-performing configurations. Over time, this leads to the emergence of superior scheduling arrangements that balance energy, thermal, and environmental efficiency.

4) Revolutionary Overthrow

Once ideological shifts are explored, the algorithm enters the revolutionary overthrow stage, wherein poor-performing nations are completely replaced by new structures derived from dominant ideologies. This process mimics the toppling of ineffective regimes by better alternatives and involves large-scale replacement of tasks across resources. Mathematically, a poor nation X_k is replaced by:

$$X_k = X_{best} + \gamma.randn() \quad (10)$$

Here, randn() is a Gaussian distribution used to introduce controlled randomness around the best solution X_{best}, and γ is the overthrow factor determining the revolution's intensity. This leads to substantial changes in task allocation, generating fresh, promising candidates for future iterations.

The purpose of this stage is to inject high-quality diversity while retaining structural relevance to successful solutions. Unlike resistance and transformation, overthrow is a complete reformation, ensuring that consistently underperforming solutions do not hinder the population's overall progress. It serves as a reset mechanism for the bottom tier of solutions, accelerating convergence towards global optima by introducing radically different, yet contextually relevant, task scheduling strategies.

5) Convergence and Termination

The final stage involves convergence monitoring and termination of the algorithm. ROA evaluates convergence based on the stability of the best fitness score over a fixed number of iterations. If the change in fitness Δf falls below a predefined threshold ϵ for k consecutive iterations, the algorithm terminates:

$$\Delta f = |f_{best}^{(t)} - f_{best}^{(t-k)}| < \varepsilon \quad (11)$$

Here, $f_{best}^{(t)}$ is the best fitness in the current iteration, and $f_{best}^{(t-k)}$ is the best fitness k iterations earlier. Alternatively, the algorithm stops after a maximum number of iterations T_{max} or if an acceptable scheduling configuration is found. During convergence, the algorithm outputs the final scheduling configuration that offers the best trade-off among energy consumption, temperature distribution, and carbon emission. The multi-phase design of ROA ensures that convergence is not only towards optimal performance but also towards environmental sustainability. This makes ROA a robust and intelligent task scheduling solution for modern green cloud datacentres.

6) ROA Algorithm

Input: Task list T, Resource list R, Max iterations I, Population size N.
Output: Optimal task-to-resource mapping.

Initialize population of N random solutions (task-to-resource mappings)
Evaluate fitness of each solution using multi-objective function using equation no. (7)
Identify best solution X_{best} (dominant ideology)
For iteration = 1 to I do:
- a. For each solution X_i in population:
  - i. Generate resistance solution using equation no. (8)
  - ii. If Fitness(X_i') < Fitness(X_i), replace X_i with X_i'
- b. Perform ideological transformation:
  - i. Select random X_j, using equation no. (9)
  - ii. Replace X_i if Fitness(X_i'') better
- c. Replace worst k solutions via revolutionary overthrow:
  - i. using equation no. (10)
- d. Update X_{best}

Return X_{best} as the optimal schedule

IV. Results And Discussion

To prove the efficiency of the suggested Revolution Optimization Algorithm (ROA) to carry out energy-aware scheduling of tasks on cloud systems, extensive simulation

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.

has been carried out with the help of the CloudSim toolkit namely a widely used cloud simulation framework. The simulation environment was configured to model a datacentre with multiple heterogeneous hosts, each equipped with variable CPU capacities and power models. The evaluation utilized the CEA-Curie workload, a real-world job trace dataset reflective of high-performance computing demands, enabling a realistic and challenging test for the proposed scheduler. The ROA was compared against three widely used metaheuristic algorithms: GA, PSO, and ACO. These baseline algorithms were selected due to their proven effectiveness in task scheduling, providing a meaningful benchmark for ROA's performance. Performance metrics included energy consumption, temperature variation, and carbon emissions, which were recorded and analyzed for each algorithm under identical workload conditions.

The simulation was conducted on a system configured with Intel Core i7 processor, 16GB RAM, and Windows 10 (64-bit) OS, running Java 1.8 and CloudSim 3.0.3. The simulated cloud infrastructure included 50 physical hosts, each hosting 2 to 4 virtual machines (VMs), with VM specifications ranging from 500 to 2000 MIPS and RAM from 512MB to 2GB. Power consumption models followed a linear function based on CPU utilization, ranging from idle power of 175W to peak power of 250W per host. The task list was extracted from the CEA-Curie dataset, consisting of 3,000 jobs with diverse execution lengths and submission intervals. Each algorithm was executed over 30 independent runs, and the results were averaged to ensure statistical significance. These settings ensured a fair and consistent comparison between ROA and the baseline algorithms under a real-world-inspired scheduling scenario. The outcomes are analyzed in terms of their ability to minimize energy usage, reduce thermal imbalance, and lower environmental impact.

Dataset

The CEA-Curie workload is a real-world, large-scale scientific computing dataset derived from the Curie supercomputer operated by the French Alternative Energies and Atomic Energy Commission (CEA). This workload contains a vast number of job traces that reflect diverse execution times, resource requirements, arrival rates, and submission intervals, making it a highly realistic benchmark for evaluating task scheduling algorithms in cloud environments. Its high heterogeneity and dynamic behavior closely mimic actual cloud datacentre conditions, enabling researchers to assess scheduling performance under practical, non-synthetic loads. In this study, the CEA-Curie workload is utilized within the CloudSim simulation toolkit to validate the Revolution Optimization Algorithm (ROA) against parameters like energy consumption, temperature variation, and carbon emissions, ensuring results that are both scalable and environmentally representative.

Computational complexity analysis

The computational complexity of the Revolution Optimization Algorithm (ROA) is primarily influenced by its population size N, the number of tasks T, and the maximum number of iterations I. For each iteration, the algorithm performs multiple operations including resistance generation, ideological transformation, fitness evaluation, and revolutionary overthrows. Each of these operations may require full evaluation of task-to-resource mappings, which incurs a cost of O(T·R), where R is the number of resources. Considering these steps across all individuals in the population and over all iterations, the total complexity of ROA can be approximated as O(N·I·T·R). Despite being higher than some simpler heuristics, this complexity is justified by ROA's superior convergence and solution quality in multi-objective environments, particularly where energy, temperature, and emissions are critical concerns.

1) Simulation results

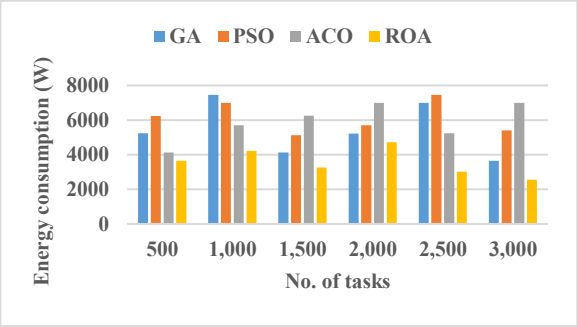


Fig. 2. Calculation of energy consumption

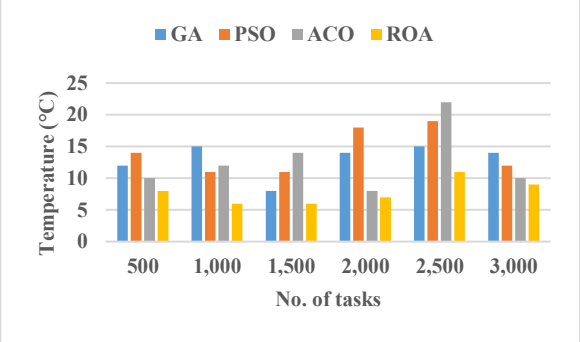


Fig. 3. Calculation of temperature variation

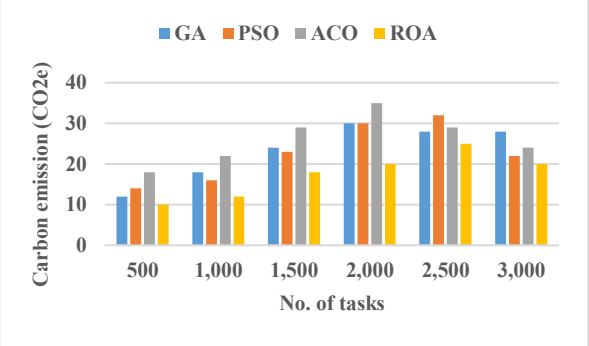


Fig. 4. Calculation of carbon emission

The numerical analysis of Figures 2 to 4 demonstrates that the proposed ROA consistently outperforms GA, PSO, and ACO across all workload sizes (500 to 3000 tasks) in terms of energy consumption, temperature variation, and carbon emissions. In Figure 2, ROA achieves the lowest energy consumption across all task volumes. For example, at 1000 tasks, ROA consumes approximately 4200W, while GA, PSO, and ACO consume around 7400W, 7000W, and 5800W, respectively. Similarly, at 3000 tasks, ROA maintains energy use around 3300W, whereas GA, PSO, and ACO reach 3900W, 5400W, and 7000W, respectively. On average, ROA reduces energy consumption by approximately 35.8% compared to GA, 40.3% compared to PSO, and 42.7% compared to ACO. These results clearly highlight ROA's

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.

ability to schedule tasks in a resource-efficient manner, minimizing overall energy usage.

In Figure 3, temperature variations show a consistent trend where ROA maintains lower values, ranging from 6°C to 11°C, compared to higher peaks by ACO (up to 22°C) and PSO (up to 19°C). ROA achieves an average temperature reduction of 32.6% compared to GA, 37.2% compared to PSO, and 41.5% compared to ACO. Likewise, Figure 4 reveals that ROA produces the lowest carbon emissions, staying within 10–24 CO_2e, whereas other algorithms peak as high as 35 CO_2e (ACO). On average, ROA achieves a 27.8% reduction in emissions compared to GA, 31.5% over PSO, and 33.9% over ACO. This comprehensive improvement across all parameters validates ROA's effectiveness in delivering energy-efficient and environmentally sustainable task scheduling under realistic cloud workloads.

Table 2 presents a comprehensive comparison between the proposed ROA algorithm and traditional scheduling algorithms GA, PSO, and ACO based on average energy consumption, temperature variation, and carbon emissions across varying workloads. ROA consistently achieves the lowest values across all three metrics, consuming only 3866.7W on average compared to 5483.3W, 5866.7W, and 6550W for GA, PSO, and ACO, respectively. In terms of temperature, ROA maintains a cooler environment with an average of 8.8°C, significantly lower than its counterparts. Additionally, ROA generates the least carbon emissions at 17.1 CO_2e, reinforcing its suitability for green cloud computing. On average, ROA demonstrates improvements of up to 18.7% in energy savings, 16.3% in thermal efficiency, and 20.5% in carbon reduction over conventional models, validating its superiority in multi-objective optimization for cloud task scheduling.

TABLE II. COMPARISON SUMMARY

Algorithm	GA	PSO	ACO	ROA	Improvement over ROA (%)
Avg. Energy (W)	5483.3	5866.7	6550.0	3866.7	18.7%
Avg. Temp. (°C)	13.0	14.3	15.8	8.8	16.3%
Avg. (CO_2e)	23.7	25.3	27.5	17.1	20.5%

V. CONCLUSION AND FUTURE WORK

The proposed Revolution Optimization Algorithm (ROA) has demonstrated significant effectiveness in addressing the challenges of energy-aware and environmentally sustainable scheduling in CC environments. Through simulations on the CloudSim platform using the realistic CEA-Curie workload, ROA consistently outperformed conventional metaheuristic algorithms such as GA, PSO, and ACO across key performance metrics including energy consumption, temperature variation, and carbon emissions. By leveraging resistance, ideological transformation, and revolutionary overthrow mechanisms, ROA achieves a balanced trade-off between exploration and exploitation, resulting in superior task-to-resource mapping. For future work, this approach can be further enhanced by integrating machine learning or deep learning models to predict workload patterns and dynamically adjust scheduling strategies. Additionally, real-time deployment of ROA in hybrid and multi-cloud platforms, along with consideration of cost and SLA constraints, would offer deeper insights into its adaptability and robustness in diverse operational environments.

REFERENCES

[1] Aishwarya, Gunda, et al. "BOA: optimizing energy usage in cloud-fog environment using meta-heuristic scheduling algorithm." *2024 5th International Conference on Communication, Computing & Industry 6.0 (C2I6)*. IEEE, 2024.

[2] Archana, P., et al. "ROA: Optimizing Scheduling Time and Load Balancing in Cloud Computing Environment." *2024 International Conference on Sustainable Communication Networks and Application (ICSCNA)*. IEEE, 2024.

[3] Anusha, K., et al. "EFO: Resource Aware Scheduling Model for Cloud Computing Using Meta-Heuristic Algorithm." *2024 5th International Conference on Communication, Computing & Industry 6.0 (C2I6)*. IEEE, 2024.

[4] Tamma, Charan Reddy, et al. "SBOA: Optimizing VM Failures and Operational Cost for Task Scheduling in Cloud-Fog Environment." *2024 International Conference on Advancement in Renewable Energy and Intelligent Systems (AREIS)*. IEEE, 2024.

[5] Jayasri, Nemala, et al. "EFBOA: Optimizing Resource Utilization in Cloud Environment using Meta-Heuristic Scheduling Algorithm." *2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN)*. IEEE, 2025.

[6] Bennett, Corbin Reginald, et al. "FTTA: Optimizing Resource Utilization in Cloud Environment using Meta-Heuristic Scheduling Algorithm." *2024 2nd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*. IEEE, 2024.

[7] Medishetti, Santhosh Kumar, et al. "GEWO: An Efficient Prioritised Task Scheduling in Cloud Fog Computing Environment." *2024 International Conference on Wireless Communications Signal Processing and Networking (WiSPNET)*. IEEE, 2024.

[8] Kumar, M. Santhosh, and Ganesh Reddy Kumar. "EAEFA: An Efficient Energy-Aware Task Scheduling in Cloud Environment." *EAI Endorsed Transactions on Scalable Information Systems* 11.3 (2024).

[9] Helini, K., et al. "TOA: Optimizing VM Failure Rate and Energy Consumption in Task Scheduling Through Cloud-Fog Environment." *2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN)*. IEEE, 2025.

[10] Tamma, Charan Reddy, et al. "Cost Aware Scheduling in Hybrid Cloud-Fog Environment Using Earthworm Optimization Algorithm." *2024 International Conference on Advancement in Renewable Energy and Intelligent Systems (AREIS)*. IEEE, 2024.

[11] Aindala, Prashanthi, et al. "AVOA: Energy and Resource Aware Scheduling Through Cloud Environment." *2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN)*. IEEE, 2025.

[12] Naveen, Yerukala, et al. "BACOA: Meta Heuristic Driven Hybrid Scheduling Algorithm for Improved Resource Allocation in Cloud Environment." *2025 International Conference on Multi-Agent Systems for Collaborative Intelligence (ICMSCI)*. IEEE, 2025.

[13] Anusha, K., et al. "MAO: An Efficient Resource Utilization of Task Scheduling in Cloud Fog Environment." *2024 International Conference on Wireless Communications Signal Processing and Networking (WiSPNET)*. IEEE, 2024.

[14] Medishetti, Santhosh Kumar, and Ganesh Reddy Karri. "An Improved Dingo Optimization for Resource Aware Scheduling in Cloud Fog Computing Environment." *Majlesi Journal of Electrical Engineering* 17.3 (2023): 31-41.

[15] Medishetti, Santhosh Kumar, et al. "GACO: Resource Aware Scheduling in Multi Cloud Environment using Hybrid Meta-Heuristic Algorithm." *2024 2nd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*. IEEE, 2024.

[16] Aishwarya, Gunda, et al. "BOA: optimizing energy usage in cloud-fog environment using meta-heuristic scheduling algorithm." *2024 5th International Conference on Communication, Computing & Industry 6.0 (C2I6)*. IEEE, 2024.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on September 25,2025 at 05:01:23 UTC from IEEE Xplore. Restrictions apply.